



Escuela de Ingeniería en Computación
Ingeniería en Computación
IC6600 - Principios de Sistemas Operativos,
Grupo 40

Compresor de Archivos Basado en el Algoritmo de Huffman

Implementaciones Serial, Paralela con `fork()` y
Concurrente con `pthread`

Edgar André Araya Vargas
andrearaya1234@estudiantec.cr
2020142856

San José, Costa Rica
Setiembre 2026

Índice general

1. Introducción	2
1.1. Reseña histórica y explicación del algoritmo de Huffman	2
1.2. Definición de Paralelismo y Concurrencia	3
2. Desarrollo	5
2.1. Guía paso a paso: instalación y uso de la interfaz	5
2.1.1. Paso 1. Abrir la terminal e instalar dependencias	5
2.1.2. Paso 2. Guardar y extraer el paquete	6
2.1.3. Paso 3. Compilar los cuatro programas	7
2.1.4. Paso 4. Preparar el directorio de entrada	7
2.1.5. Paso 5. Abrir la ventana y elegir el corpus	8
2.1.6. Paso 6. Ejecutar las tres variantes y leer la tabla	8
2.1.7. Paso 7. Identificar los archivos producidos	8
2.2. Implementación del algoritmo de Huffman	9
2.3. Versión con procesos: <code>fork()</code> , IPC y sincronización	11
2.4. Versión con hilos: <code>pthread</code> y memoria compartida	12
2.5. Comparación y verificación empírica	14
2.5.1. Evidencia del corpus y del contenedor	14
2.5.2. Cómo obtiene las medidas la interfaz	15
2.5.3. Corrida completa de la interfaz	16
3. Conclusiones	17

Capítulo 1

Introducción

Este informe documenta el desarrollo del Proyecto Programado del curso IC6600 - Principios de Sistemas Operativos, correspondiente a la implementación de un compresor y descompresor de archivos basado en el algoritmo de Huffman, desarrollado en tres variantes: una versión serial, una versión paralela utilizando `fork()` e IPC, y una versión concurrente utilizando `pthread` y memoria compartida. Las tres variantes se integran en una interfaz gráfica que permite comparar su desempeño de forma empírica sobre un corpus común de cien libros descargados de Project Gutenberg a partir de su clasificación de libros más descargados en los últimos 30 días (Project Gutenberg, s.f.).

1.1. Reseña histórica y explicación del algoritmo de Huffman

El algoritmo que hoy se conoce como codificación de Huffman surgió en 1951, como resultado de un trabajo de curso en el Instituto Tecnológico de Massachusetts (MIT). El profesor Robert M. Fano ofreció a los estudiantes de un curso de posgrado sobre teoría de la información la opción de presentar un examen final o desarrollar un trabajo de investigación. David A. Huffman, entonces estudiante, optó por el trabajo de investigación, que consistía en encontrar el código binario más eficiente para representar un conjunto de símbolos según su frecuencia de aparición (Stix, 1991). Tras varios meses de trabajo sin lograr una solución satisfactoria, Huffman encontró un método constructivo que resultó ser más eficiente que el algoritmo de Shannon-Fano, el método de referencia hasta ese momento (Stix, 1991). El resultado se publicó al año siguiente bajo el título *A Method for the Construction of Minimum-Redundancy Codes* (Huffman, 1952).

El problema que resuelve el algoritmo es el siguiente: dado un conjunto finito de símbolos, cada uno con una frecuencia (o probabilidad) de aparición conocida, se busca asignar a cada símbolo un código binario de largo variable, de forma que el largo promedio del mensaje codificado sea el menor posible (Huffman,

1952). La idea central es que los símbolos más frecuentes reciban códigos más cortos, y los símbolos menos frecuentes reciban códigos más largos.

El algoritmo de Huffman construye este código mediante un árbol binario, siguiendo un procedimiento incremental (voraz o *greedy*):

1. Se calcula la frecuencia de aparición de cada símbolo en los datos de entrada.
2. Se crea un nodo hoja por cada símbolo, con su frecuencia asociada, y se colocan todos los nodos en una cola de prioridad ordenada por frecuencia ascendente.
3. Se extraen los dos nodos de menor frecuencia y se combinan en un nuevo nodo interno, cuya frecuencia es la suma de las frecuencias de sus dos hijos. Este nuevo nodo se reinserta en la cola de prioridad.
4. El paso anterior se repite hasta que queda un único nodo en la cola: la raíz del árbol.
5. El código binario de cada símbolo se obtiene recorriendo el árbol desde la raíz hasta la hoja correspondiente, asignando un bit (0 o 1) a cada rama recorrida.

Este procedimiento garantiza que ningún código sea prefijo de otro código (propiedad de *prefix-free code*), lo que permite decodificar el mensaje comprimido de forma no ambigua, sin necesidad de separadores entre símbolos (Huffman, 1952). Además, el árbol resultante minimiza el largo promedio ponderado del código, por lo que se considera un código de *redundancia mínima* (Huffman, 1952).

En este proyecto se aplica la codificación directamente a los bytes de cada archivo, de forma que la representación del texto no depende del idioma ni del conjunto de caracteres empleado.

1.2. Definición de Paralelismo y Concurrency

Aunque en el lenguaje cotidiano los términos *paralelismo* y *concurrency* suelen usarse como sinónimos, en el contexto de los sistemas operativos designan conceptos distintos.

La **concurrency** es una propiedad de un sistema que permite que múltiples actividades estén en progreso durante un mismo intervalo de tiempo, sin que necesariamente se estén ejecutando en el mismo instante físico (Silberschatz et al., 2018). En un sistema con un único núcleo de procesamiento, la concurrency se logra mediante cambios de contexto: el planificador del sistema operativo alterna rápidamente la ejecución entre distintos procesos o hilos, dando la impresión de que avanzan simultáneamente, aunque en realidad solo uno ocupa el procesador en cada instante (Silberschatz et al., 2018). La concurrency es,

entonces, una propiedad conceptual o estructural del programa: describe cómo se organiza el trabajo en unidades que pueden intercalarse.

El **paralelismo**, en cambio, ocurre cuando dos o más actividades se ejecutan literalmente al mismo tiempo, lo cual solo es posible si el sistema cuenta con más de una unidad de procesamiento física (múltiples núcleos o múltiples procesadores) (Silberschatz et al., 2018). A diferencia de la concurrencia, el paralelismo es una propiedad del tiempo de ejecución (*runtime*): depende de que el hardware subyacente sea capaz de sostener varias líneas de ejecución de forma simultánea (Silberschatz et al., 2018). Un sistema concurrente se vuelve efectivamente paralelo únicamente cuando existen suficientes núcleos disponibles para ejecutar sus procesos o hilos al mismo tiempo (Silberschatz et al., 2018).

En ambos casos, cuando varias actividades comparten datos o recursos, es necesario coordinar su ejecución mediante mecanismos de sincronización, para evitar condiciones de carrera y garantizar la consistencia de los datos compartidos (Silberschatz et al., 2018).

Esta distinción es relevante para este proyecto porque las tres versiones del compresor exploran puntos distintos de este espectro:

- La **versión serial** no introduce concurrencia ni paralelismo: los archivos se procesan uno a la vez, en un único flujo de ejecución.
- La **versión con `fork()`** introduce paralelismo real cuando la máquina cuenta con múltiples núcleos: cada proceso hijo es una réplica independiente del proceso padre, con su propio espacio de memoria, por lo que requiere una estrategia explícita de comunicación entre procesos (IPC) para intercambiar resultados.
- La **versión con `pthread`** introduce concurrencia mediante múltiples hilos que comparten el mismo espacio de memoria del proceso, lo que exige mecanismos de sincronización (como mutex o semáforos) para coordinar el acceso a los datos compartidos, y que se traduce en paralelismo real si el sistema operativo distribuye esos hilos entre distintos núcleos.

Capítulo 2

Desarrollo

2.1. Guía paso a paso: instalación y uso de la interfaz

El proyecto se entrega como un único archivo ZIP:

```
proyecto1-compresor-unico.zip
```

Al extraerlo se crea el directorio `proyecto1-compresor/`, con el código C, el archivo `Makefile`, el script de descarga y los archivos fuente del informe. La ejecución de las tres variantes y de la interfaz gráfica se verificó en Debian GNU/Linux 13 con GNOME. El ZIP contiene el código fuente y se adjunta junto al PDF. Para satisfacer también el requisito del enlace al código fuente, la dirección pública del repositorio se incorporará a este apartado antes de presentar la entrega.

2.1.1. Paso 1. Abrir la terminal e instalar dependencias

Abrir *Terminal* desde el menú de GNOME. Iniciar una sesión de administrador con el comando siguiente; introducir la contraseña de **root** cuando se solicite. La contraseña del usuario ordinario no sustituye a la de root.

```
su -
```

Confirmar que el indicador de la terminal corresponde a root (por ejemplo, `root@debian: #`). Copiar el bloque siguiente en esa misma terminal. Primero actualiza la información de los paquetes; después instala únicamente lo necesario para compilar y ejecutar los cuatro programas. No se instalan herramientas de LaTeX porque el PDF ya forma parte de la entrega.

```

apt update
apt install -y build-essential
apt install -y pkg-config
apt install -y libssl-dev
apt install -y libgtk-4-dev
apt install -y wget ca-certificates
apt install -y python3 xdg-user-dirs
exit
id -un

```

Esperar a que termine `apt install` antes de escribir `exit`. Ese comando sale de root y devuelve a la sesión de usuario ordinario; `id -un` debe mostrar el nombre de ese usuario (`oper` en la máquina de prueba). **Si muestra root, no continuar: salir de la sesión administrativa con `exit` y repetir `id -un`.** La compilación y la GUI se ejecutan como usuario ordinario; no es necesario agregarlo al grupo `sudo`.

Comprobación del paso 1. La terminal debe volver a mostrar el usuario ordinario (en la instalación de prueba, `oper`), y los comandos `gcc --version`, `pkg-config --modversion gtk4` y `pkg-config --modversion openssl` deben mostrar versiones, sin mensajes de «comando no encontrado».

2.1.2. Paso 2. Guardar y extraer el paquete

Descargar el archivo adjunto `proyecto1-compresor-unico.zip` desde la plataforma de entrega. En el diálogo del navegador seleccionar *Guardar archivo* y elegir la carpeta *Descargas*. Abrir *Archivos* de GNOME, entrar en *Descargas* y verificar visualmente que el nombre sea `proyecto1-compresor-unico.zip`. Si el navegador añadió un sufijo por existir una descarga anterior, usar *Cambiar nombre* en *Archivos* para darle exactamente el nombre indicado. No abrir el ZIP desde el explorador: los comandos siguientes lo extraen en la ubicación esperada.

Volver a la terminal del usuario ordinario y ejecutar este bloque para comprobar el ZIP, extraerlo y comprobar los archivos fuente:

```

DESCARGAS="$(xdg-user-dir DOWNLOAD)"
ARCHIVO="$DESCARGAS/proyecto1-compresor-unico.zip"
ls -lh "$ARCHIVO"
python3 -m zipfile -e "$ARCHIVO" "$HOME"
cd "$HOME/proyecto1-compresor"
pwd
ls -l Makefile
ls -l src/serial/serial.c
ls -l src/fork/fork.c
ls -l src/pthread/pthread.c
ls -l src/gui/gui.c

```

`pwd` debe mostrar `/home/oper/proyecto1-compresor` en la máquina de prueba, o la ruta equivalente bajo la carpeta personal de otro usuario. La lista debe mostrar los cuatro archivos fuente (`serial`, `fork`, `pthread` y `GUI`) y el `Makefile`. **Si no aparece `src/gui/gui.c`, detenerse: se extrajo un ZIP anterior o incompleto.**

2.1.3. Paso 3. Compilar los cuatro programas

Ejecutar los siguientes comandos en una terminal de usuario ordinario:

```
cd "$HOME/proyecto1-compresor"
make
ls -l bin/serial bin/fork bin/pthread bin/gui
```

El primer comando entra al proyecto desde cualquier carpeta. `make` debe terminar sin errores y `ls` debe mostrar cuatro ejecutables. Si falta uno, revisar el error de `make` antes de continuar.

2.1.4. Paso 4. Preparar el directorio de entrada

Si `corpus/` ya contiene los libros que se desean probar, continuar con el paso 5. Si es una instalación nueva y todavía no hay libros, ejecutar:

```
cd "$HOME/proyecto1-compresor"
bash scripts/descargar_corpus.sh
find corpus -maxdepth 1 -type f -name '*.txt' | wc -l
```

La selección se obtiene de la página oficial de Project Gutenberg que publica los libros más descargados en los últimos 30 días (Project Gutenberg, s.f.). La clasificación cambia con el tiempo y algunas descargas pueden faltar por disponibilidad o red. El script registra los identificadores consultados en `scripts/ids_30dias.txt` y los resultados en `scripts/descarga.log`. Para reproducir exactamente las cifras de este informe se necesitan los mismos 100 archivos verificados en la ejecución original; una descarga posterior podría producir otra selección. La interfaz también acepta un directorio propio de archivos regulares, siempre que el usuario tenga permiso de lectura y espacio para los resultados.

Comprobación del paso 4. Esperar a que termine el script antes de leer el resultado de `find`. En la ejecución documentada se obtuvieron 100 archivos `.txt`. Si aparecen menos de 100, consultar `scripts/descarga.log`; ese resultado no reproduce el corpus medido aquí.

2.1.5. Paso 5. Abrir la ventana y elegir el corpus

En la sesión GNOME del usuario ordinario, ejecutar:

```
cd "$HOME/proyecto1-compresor"  
./bin/gui
```

Se abrirá la ventana *Compresor Huffman — comparación*. Mantener abierta la terminal. Pulsar *Elegir carpeta de archivos originales*; en el selector de GNOME, abrir *Carpeta personal*, entrar en **proyecto1-compresor**, elegir **corpus** y confirmar. En la máquina de prueba, la ruta seleccionada es:

```
/home/oper/proyecto1-compresor/corpus
```

Debajo del botón debe aparecer la ruta elegida; verificar que termine en */corpus* antes de iniciar la prueba. La ventana contiene otro botón cuyo texto es *Comprimir, descomprimir y comparar (4 trabajadores)*. La tabla de resultados de la Sección 2.5 muestra los valores observados en la ejecución verificada.

2.1.6. Paso 6. Ejecutar las tres variantes y leer la tabla

Pulsar *Comprimir, descomprimir y comparar (4 trabajadores)* **una sola vez**. Aparecerá un mensaje que pide esperar; no cerrar la ventana mientras se procesan los archivos. La aplicación ejecuta compresión y descompresión para **serial**, **fork** y **pthread**, y coteja los archivos de salida con los originales. En el corpus de 100 libros puede tardar; el botón queda deshabilitado hasta que termina.

Al finalizar, la tabla debe tener tres filas, una por variante. Revisar las columnas *Estado*, *Salud MD5*, *Comprimir (s)*, *Descomprimir (s)*, *Mejora C / D*, *Original (bytes)*, *HUF (bytes)* y *Razón original/HUF*. Para una ejecución satisfactoria, *Estado* muestra *Correcto*, y la salud muestra el cociente de firmas verificadas sobre los archivos de entrada. La mejora de la fila serial aparece como una raya porque es la referencia. La ventana tiene desplazamiento horizontal: si alguna columna no se ve, maximizar la ventana o desplazarse a la derecha, sin repetir la prueba.

Los resultados completos de esa ejecución se consignan en la Sección 2.5.

2.1.7. Paso 7. Identificar los archivos producidos

Leer el texto bajo la tabla: indica la carpeta nueva donde se guardaron los resultados, dentro del proyecto. Su nombre empieza por **comparacion-** y termina en seis caracteres únicos. Para localizarla, abrir *Archivos* desde el menú de GNOME, pulsar *Carpeta personal*, abrir **proyecto1-compresor** y después la carpeta **comparacion-** cuyo nombre completo coincida con el mostrado bajo la tabla. Allí deben aparecer **serial.huf**, **fork.huf** y **pthread.huf**, junto con tres

carpetas llamadas `serial`, `fork` y `pthread` con los archivos recuperados. Cada ejecución crea una carpeta nueva; los resultados anteriores no se reemplazan.

La versión de línea de comandos sigue disponible cuando se desee comprimir o expandir un contenedor individual; las instrucciones de uso se muestran al ejecutar `bin/serial`, `bin/fork` o `bin/pthread` sin argumentos. No se requiere invocarlas para obtener la tabla de la interfaz.

2.2. Implementación del algoritmo de Huffman

Las tres variantes utilizan la misma representación de datos y las mismas funciones de codificación y decodificación. Estas funciones están implementadas en `src/serial/serial.c`. La unidad de trabajo es cada archivo regular situado directamente en el directorio de entrada. Los nombres se ordenan mediante `strcmp` antes de procesarlos; esta decisión permite que el contenedor tenga un orden reproducible aun cuando el sistema de archivos enumere sus entradas de otra forma. El programa procesa bytes, no caracteres Unicode: construye un alfabeto de 256 valores posibles y cuenta cuántas veces aparece cada valor en el archivo. Así se conservan exactamente los bytes originales, incluidos los saltos de línea y la codificación del texto.

Para construir el árbol, `build` crea una hoja por cada byte cuya frecuencia sea mayor que cero. En cada iteración busca por recorrido los dos nodos activos con menor frecuencia, los une y asigna al nodo padre la suma de ambas frecuencias. Si hay frecuencias iguales, desempata usando el menor byte presente en cada subárbol. Repite la operación hasta obtener la raíz. Esta es la construcción de códigos binarios de longitud variable descrita por Huffman (Huffman, 1952); el desempate especificado en nuestro código permite reconstruir exactamente el mismo árbol durante la descompresión. La búsqueda de los mínimos recorre como máximo 511 nodos por iteración y el alfabeto está limitado a 256 bytes. En este proyecto se prefirió esa implementación directa, sin una cola de prioridad adicional.

Fragmento del código real, `src/serial/serial.c`: el arreglo `freq` contabiliza bytes antes de construir el árbol, y `less` fija un desempate independiente del orden en que se hayan encontrado las hojas:

Listing 2.1: Conteo de frecuencias y criterio de orden del árbol

```
uint64_t freq[256]={0};
for(size_t i=0;i<n;i++)freq[data[i]]++;

static int less(const Node *a,const Node *b){
    if(a->freq!=b->freq)return a->freq<b->freq;
    return a->min_symbol<b->min_symbol;
}
```

La función **makecodes** recorre el árbol y asigna cero al hijo izquierdo y uno al derecho. **packone** sustituye cada byte de entrada por los bits de su código, coloca primero el bit más significativo de cada byte comprimido y rellena con ceros el último byte cuando faltan bits para completarlo. Un archivo vacío produce cero bytes de datos codificados. Si el archivo contiene un único valor de byte distinto, se le asigna el código de un bit cero. Estas dos situaciones reciben tratamiento explícito también en el decodificador.

Se definió un formato propio llamado **HUF1**, compartido por las tres variantes. Cada entero se escribe en orden de bytes *little endian*; la longitud del nombre ocupa dos bytes, la cantidad de entradas cuatro y los tamaños ocho. La tabla de frecuencias tiene 256 enteros de ocho bytes, incluso cuando algunas frecuencias sean cero. Por tanto, cada entrada tiene 2048 bytes de tabla, además del nombre y los demás metadatos; ese costo fijo reduce la eficiencia en archivos pequeños.

Campo del contenedor	Extensión
Firma HUF1; cantidad de archivos	4 bytes; 4 bytes
Longitud del nombre; nombre	2 bytes; longitud indicada
Tamaño original; resumen MD5	8 bytes; 16 bytes
Frecuencias de los 256 valores	256×8 bytes
Longitud de los datos; datos comprimidos	8 bytes; longitud indicada

Antes de codificar cada archivo se calcula el MD5 de sus bytes originales mediante las funciones **EVP_DigestInit_ex**, **EVP_DigestUpdate** y **EVP_DigestFinal_ex** de OpenSSL y se guarda su resumen de 16 bytes en la entrada correspondiente (OpenSSL Project, s.f.-a, s.f.-b). Durante la descompresión, **unpackone** lee el nombre, tamaño, resumen y frecuencias; verifica que las frecuencias sumen el tamaño declarado, reconstruye el árbol con las mismas reglas, recupera exactamente esa cantidad de bytes y comprueba el MD5 antes de crear el archivo de salida. También comprueba que el relleno final sea cero y rechaza nombres con separadores de rutas o equivalentes a los directorios actual y padre. El modo **wbx** combina escritura binaria con el modificador exclusivo **x**; en C11, este modificador hace que la apertura falle si el archivo ya existe, evitando su sobrescritura (cppreference.com, s.f.). MD5 cumple aquí el requisito de detección de cambios del proyecto; la validación externa con SHA-256 del corpus se realizó además para comprobar que los archivos restaurados coincidieran byte por byte con el manifiesto previo.

Fragmento del código real: las líneas siguientes pertenecen a `packone` y `unpackone`, respectivamente. La primera guarda la firma original antes de la secuencia codificada; la segunda rehace el resumen de los bytes recuperados y lo compara con el almacenado antes de abrir el archivo de destino:

Listing 2.2: Almacenamiento y comprobación de la firma MD5

```
unsigned char md5[16];digest(data,n,md5);
putnum(out,strlen(name),2);writeall(out,name,strlen(name));
putnum(out,n,8);
writeall(out,md5,16);

digest(data,(size_t)n,actual);
check(memcmp(actual,expected,16)==0,"MD5 no coincide");
```

El resultado de una compresión incluye una entrada por archivo. La línea final de una corrida serial sobre el corpus registrado fue **Total comprimido: 100 archivos;** tras descomprimir, se obtuvo **Total descomprimido y verificado: 100 archivos.** Estas líneas son salidas del programa, no valores supuestos del algoritmo.

2.3. Versión con procesos: `fork()`, IPC y sincronización

`src/fork/fork.c` reutiliza `packone` y `unpackone` de la implementación serial. El padre obtiene primero la lista ordenada de nombres y escribe la cabecera del contenedor. A continuación lanza procesos hijos en tandas cuyo tamaño máximo indica el argumento `workers`; en las mediciones se utilizaron cuatro. `fork()` crea un proceso hijo a partir del padre, y `pipe()` proporciona un canal unidireccional para comunicar el resultado de su tarea (The Linux man-pages Project, 2026, s.f.-a). Cada hijo comprime un archivo completo en su propio archivo temporal; el canal transmite un byte de éxito y el archivo temporal contiene los datos de la entrada del contenedor. Por tanto, los datos voluminosos se intercambian mediante archivos temporales y la tubería transporta únicamente el estado de terminación.

El padre cierra los extremos de las tuberías que no utiliza, lee la señal de cada hijo y espera su finalización mediante `waitpid()` (The Linux man-pages Project, s.f.-d). Solo si recibe la señal esperada y el proceso termina correctamente abre los archivos temporales y concatena las entradas al contenedor en el orden original de nombres. Luego elimina cada temporal. Antes de crear los hijos, vacía el búfer de salida del archivo de la cabecera; así evita que la copia de ese búfer heredada después de `fork()` duplique bytes. La regla de un temporal por tarea evita que varios hijos escriban sobre la misma posición del contenedor. La espera por tanda limita el número de procesos activos y hace que la inte-

gración empiece cuando han terminado las tareas de esa tanda. Este esquema puede dejar núcleos ociosos cuando una tarea de la tanda tarda mucho más que las demás, pero mantiene un orden de ensamblado verificable.

Fragmento de `src/fork/fork.c`: el padre crea una tubería antes de cada `fork`; cada hijo recibe un archivo distinto y el extremo de escritura de su tubería. `finish` recibe el byte de estado y espera al proceso antes de incorporar su resultado:

Listing 2.3: Creación de hijos y sincronización por tubería

```
int p[2];check(pipe(p)==0,"crear pipe");
jobs[j].temp=temporary(tmpdir,i+j);
pid_t pid=fork();check(pid>=0,"crear proceso hijo");
if(pid==0){
    close(p[0]);close(fileno(out));
    child_compress(dir,names[i+j],jobs[j].temp,p[1]);
}
close(p[1]);jobs[j].pid=pid;jobs[j].fd=p[0];

finish(jobs,batch);
```

La sincronización no supone que el primer hijo iniciado termine primero: `finish` verifica cada resultado y después el padre ensambla las entradas por índice. En la prueba realizada, el contenedor `fork.huf` coincidió byte por byte con `serial.huf`, y los 100 archivos restaurados coincidieron con el manifiesto SHA-256.

Para descomprimir, el padre recorre primero el contenedor y registra la posición inicial de cada entrada. Después asigna posiciones distintas a los hijos, también por tandas. Cada hijo abre el contenedor con su propio descriptor, se desplaza a su posición, descomprime la entrada, verifica el MD5 y crea el archivo de destino. Abrir el contenedor por separado evita que los hijos interfieran al cambiar la posición de lectura. Las tuberías y `waitpid()` comunican y confirman el estado de las tareas. Como cada hijo crea un archivo diferente, la escritura de los resultados no requiere un bloqueo compartido. Si una tarea falla, el proceso principal termina con error; la implementación no promete una reversión automática de los archivos de salida que ya se hayan creado.

2.4. Versión con hilos: `pthread` y memoria compartida

`src/pthread/pthread.c` usa las mismas funciones de compresión y descompresión que las otras variantes. Aquí los trabajadores son hilos de un mismo proceso: comparten direcciones de memoria, incluida una estructura `Shared` con un índice de próxima tarea, nombres de archivos, posiciones en el contenedor y espacios para resultados (The Linux man-pages Project, s.f.-c). `run_workers`

crea el número solicitado de hilos con `pthread_create` y espera a todos con `pthread_join`. En las comprobaciones realizadas se utilizaron cuatro hilos.

Cada trabajador solicita una tarea mediante `take`: toma el mutex, lee e incrementa el índice de tarea y libera el mutex. Esta sección crítica impide que dos hilos reclamen el mismo índice. El cálculo de frecuencias, la construcción del árbol, la codificación y el cálculo del resumen ocurren después de liberar el mutex; por eso el trabajo por archivo puede avanzar en paralelo. La protección con `pthread_mutex_lock` y `pthread_mutex_unlock` corresponde al acceso al índice compartido, no a un bloqueo global de la compresión (POSIX Programmer's Manual, s.f.).

Fragmento de `src/threads/threads.c`: el incremento protegido asigna un índice único; `pthread_join` impide que se ensamble el archivo mientras algún trabajador siga escribiendo su casilla de resultados:

Listing 2.4: Reparto de tareas y espera de hilos

```
static size_t take(Shared *s){
    check(pthread_mutex_lock(&s->lock)==0,"bloquear cola");
    size_t i=s->next++;
    check(pthread_mutex_unlock(&s->lock)==0,"desbloquear cola");
    return i;
}

for(unsigned j=0;j<count;j++)
    check(pthread_create(&threads[j],NULL,worker,s)==0,"crear hilo");
for(unsigned j=0;j<count;j++)
    check(pthread_join(threads[j],NULL)==0,"esperar hilo");
```

Durante la compresión, cada hilo escribe la entrada asignada en un búfer de memoria propio abierto con `open_memstream` (The Linux man-pages Project, s.f.-b). Al terminar, publica el búfer y su longitud en la casilla correspondiente a su índice. Esas casillas son distintas para cada tarea. Una vez unidos todos los hilos, el hilo principal escribe la cabecera y concatena los resultados por índice, conservando el mismo orden del contenedor serial. La sincronización mediante `pthread_join` garantiza que el ensamblado se realice después de que terminen los trabajadores (The Linux man-pages Project, s.f.-c).

En la descompresión se indexan primero las posiciones de todas las entradas. Cada hilo toma un índice bajo el mismo mutex, abre su propio flujo de lectura del contenedor, se desplaza a la entrada asignada y ejecuta `unpackone`. Los flujos independientes evitan compartir una posición de lectura mutable. Cada hilo escribe un archivo de salida distinto, y `unpackone` comprueba el MD5 antes de crearlo. El enfoque evita archivos temporales para la compresión, a costa de conservar en memoria las entradas comprimidas hasta el ensamblado final. Al igual que en la versión con procesos, los errores no implican una reversión

automática de las salidas ya creadas.

En el corpus de prueba, el resultado `pthread.huf` fue idéntico a los contenedores `serial` y `fork`, y se restauraron 100 de 100 archivos con SHA-256 coincidente. Esta prueba confirma el resultado obtenido con cuatro hilos en ese corpus; no equivale a una prueba exhaustiva de todos los posibles archivos de entrada.

2.5. Comparación y verificación empírica

Se utilizaron 100 archivos del corpus descargado, con un tamaño original agregado de 82 287 268 bytes. Los tres contenedores generados en las pruebas de terminal midieron 48 207 420 bytes cada uno y fueron idénticos al compararlos byte por byte. Después de descomprimirlos se cotejaron los 100 archivos con el manifiesto SHA-256 previo: coincidieron 100 de 100 en cada variante. La nueva corrida de la interfaz confirmó esos tamaños para las tres variantes, además de mostrar la salud MD5 de cada una en la misma tabla. La razón original/contenedor fue $82\,287\,268/48\,207\,420 \approx 1,707 : 1$ y la reducción de tamaño calculada fue 41,42 %.

2.5.1. Evidencia del corpus y del contenedor

Antes de comprimir, la comprobación de existencia y tamaño del corpus mostró **Faltantes: 0; vacíos: 0**. La suma de los tamaños de los archivos originales y la validación del manifiesto produjeron estas salidas:

Listing 2.5: Salidas obtenidas al comprobar el corpus

```
Archivos: 100; bytes originales: 82287268
SHA-256: 100 archivos verificados
```

El manifiesto `resultados/corpus_sha256.txt` contiene 100 líneas. Se generó antes de comprimir y se aplicó a los directorios de salida después de descomprimir. Las salidas resumidas observadas para cada variante fueron:

Listing 2.6: Salidas verificadas de los tres programas

```
Total comprimido: 100 archivos
Total descomprimido y verificado: 100 archivos
Corpus restaurado: 100/100 SHA-256 coinciden

Total comprimido con fork: 100 archivos
Total descomprimido y verificado con fork: 100 archivos
Corpus restaurado con fork: 100/100 SHA-256 coinciden

Total comprimido con pthread: 100 archivos
```

```
Total descomprimido y verificado con pthread: 100 archivos
Corpus restaurado con pthread: 100/100 SHA-256 coinciden
```

En las tres compresiones se midieron 48207420 bytes comprimidos. También se compararon los contenedores con `cmp: fork.huf` y `pthread.huf` coincidieron con `serial.huf`. El MD5 interno se comprobó al extraer cada entrada; el SHA-256 externo confirmó el resultado contra la copia de referencia del corpus. La igualdad de los contenedores comprueba el formato común y su orden de ensamblado en este conjunto de prueba.

Antes de utilizar la GUI se cronometó también una ejecución de cada binario en la terminal. Esas observaciones independientes fueron: `serial` 1,544 s para comprimir y 2,321 s para descomprimir; `fork` 0,956 s y 1,285 s; `pthread` 0,637 s y 0,803 s, respectivamente. No se promedian con los tiempos posteriores de la GUI: se obtuvieron en corridas distintas y con envolturas de medición diferentes. Se conservan para mostrar que las variantes se probaron de forma independiente antes de integrarlas en la ventana.

2.5.2. Cómo obtiene las medidas la interfaz

La GUI no reimplementa el algoritmo: lanza cada uno de los tres ejecutables existentes, espera su término y mide el tiempo transcurrido. Este fragmento de `src/gui/gui.c` muestra el inicio de una ejecución y el reloj monótonico empleado:

Listing 2.7: Lanzamiento y medición de los ejecutables

```
GSubprocess *process = g_subprocess_new(
    G_SUBPROCESS_FLAGS_STDOUT_PIPE |
    G_SUBPROCESS_FLAGS_STDERR_PIPE,
    &error, binary, action, input, output,
    workers ? number : NULL, NULL);
gint64 start = g_get_monotonic_time();
char *out = NULL, *err = NULL;
gboolean completed = g_subprocess_communicate_utf8(
    process, NULL, NULL, &out, &err, &error);
*seconds = (g_get_monotonic_time() - start) / 1000000.0;
```

`g_subprocess_new` crea el subproceso con los argumentos y banderas indicados (GNOME Project, s.f.-c); `g_subprocess_communicate_utf8` intercambia la entrada/salida y espera a que finalice la comunicación con el subproceso (GNOME Project, s.f.-b). El reloj monótonico de GLib devuelve microsegundos y está diseñado para medir intervalos sin verse afectado por cambios del reloj de pared (GNOME Project, s.f.-a). En esta implementación, el reloj comienza después de crear el subproceso y el intervalo termina después de recibir sus salidas y esperar su finalización; por eso las cifras de la GUI incluyen la espera y

comunicación con el proceso, pero no todo su costo de creación. El programa descomprime el contenedor de cada variante y comprueba MD5 internamente; la GUI verifica además que cada archivo restaurado sea idéntico byte por byte al de entrada. La salud se presenta como **verificados/total** y como porcentaje. Si una variante falla, su estado se muestra como *Error* y el mensaje inferior explica la causa.

2.5.3. Corrida completa de la interfaz

En la corrida verificada se seleccionó la carpeta **corpus** del proyecto y se utilizaron cuatro trabajadores en **fork** y **pthread**. Las tres variantes mostraron *Correcto* y 100/100 (100,0 %) en *Salud MD5*. Los tiempos y porcentajes que la interfaz presentó fueron:

Variante	Compresión (s)	Descompresión (s)	Mejora C	Mejora D
Serial	1,290	2,173	—	—
fork	1,066	1,335	17,4 %	38,5 %
pthread	0,591	0,766	54,2 %	64,7 %

La interfaz también mostró 82287268 en *Original (bytes)*, 48207420 en *HUF (bytes)* y 1,707:1 en *Razón original/HUF*, para cada variante. Los contenedores y directorios restaurados de esta corrida quedaron bajo la ruta que aparece al pie de la ventana; el estado *Correcto* corresponde a las verificaciones realizadas por la interfaz en esa ejecución.

La columna de mejora se obtiene mediante $100(1 - t_{\text{variante}}/t_{\text{serial}})$ para cada operación. La fila serial no tiene porcentaje porque es la referencia. La GUI calcula con los tiempos medidos antes de redondearlos para visualización; por eso recalculan a partir de los tres decimales impresos puede diferir en una décima de punto porcentual. Se trata de una sola corrida, no de una media ni de un intervalo de confianza. El estado *Correcto* indica que el ejecutable terminó sin error, pasó la verificación MD5 interna y que la interfaz cotejó byte por byte los archivos restaurados con los de entrada.

Capítulo 3

Conclusiones

Se desarrollaron tres implementaciones que comparten el mismo formato HUF1 y las mismas rutinas de codificación. En el corpus probado, las tres produjeron contenedores idénticos, y la comprobación posterior de los archivos restaurados fue satisfactoria para los 100 archivos. Esa igualdad permite atribuir las diferencias observadas en tiempos principalmente a la forma de organizar el trabajo por archivos; no se cambiaron ni el algoritmo ni el formato entre variantes.

La versión con procesos obtuvo, en la corrida gráfica completa, 17,4 % de mejora en compresión y 38,5 % en descompresión respecto a la serial. La versión con hilos mostró mejoras de 54,2 % y 64,7 %, respectivamente. Esta corrida confirmó también 100,0 % de salud MD5 y la misma razón de compresión, 1,707:1, en las tres variantes. Los porcentajes de aceleración describen esta ejecución concreta y no se interpretan como promedios de varias pruebas. La asignación por archivo, el tamaño desigual de los libros, el uso de archivos temporales en `fork`, las asignaciones de memoria en `pthread` y la actividad del sistema pueden modificar el resultado de otra ejecución.

La coordinación entre trabajadores fue diferente: `fork` comunicó el estado por tuberías y ensambló resultados temporales después de esperar a los hijos; `pthread` protegió con un mutex únicamente el índice de trabajo y esperó la finalización de los hilos antes de reunir los resultados. Ambas estrategias evitaron escrituras simultáneas sobre el mismo contenedor y conservaron el orden determinista de sus entradas. Las comprobaciones MD5 se realizaron antes de crear cada archivo restaurado, y las comparaciones SHA-256 externas proporcionaron una verificación independiente en el corpus probado.

La implementación actual procesa solo archivos regulares del primer nivel del directorio. Cada archivo se carga completo en memoria, los nombres de archivo deben caber en los dos bytes destinados a su longitud y se impone un límite de 1 GiB por archivo. Además, cada entrada ocupa 2048 bytes de tabla de frecuencias aun si el texto es pequeño. Estas decisiones mantienen sencillo el formato, pero restringen la escala y pueden perjudicar la compresión de archivos muy cortos. Como continuación técnica, serían pertinentes la lectura por bloques, una representación más compacta del árbol y varias repeticiones

de las mediciones bajo las mismas condiciones.

Referencias

- cppreference.com. (s.f.). *fopen*, *fopen_s*. Consultado el 24 de septiembre de 2026, desde <https://en.cppreference.com/w/c/io/fopen>
- GNOME Project. (s.f.-a). *g_get_monotonic_time*. Consultado el 24 de septiembre de 2026, desde https://docs.gtk.org/glib/func.get_monotonic_time.html
- GNOME Project. (s.f.-b). *GSubprocess.communicate_utf8*. Consultado el 24 de septiembre de 2026, desde https://docs.gtk.org/gio/method.Subprocess.communicate_utf8.html
- GNOME Project. (s.f.-c). *GSubprocess.new*. Consultado el 24 de septiembre de 2026, desde <https://docs.gtk.org/gio/ctor.Subprocess.new.html>
- Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 40(9), 1098-1101. <https://doi.org/10.1109/JRPROC.1952.273898>
- OpenSSL Project. (s.f.-a). *EVP_DigestInit: EVP Message Digest Routines*. Consultado el 24 de septiembre de 2026, desde https://docs.openssl.org/3.5/man3/EVP_DigestInit/
- OpenSSL Project. (s.f.-b). *EVP_md5: MD5 Message Digest*. Consultado el 24 de septiembre de 2026, desde https://docs.openssl.org/3.5/man3/EVP_md5/
- POSIX Programmer's Manual. (s.f.). *pthread_mutex_lock(3p)* — *POSIX Programmer's Manual*. Consultado el 24 de septiembre de 2026, desde https://man7.org/linux/man-pages/man3/pthread_mutex_lock.3p.html
- Project Gutenberg. (s.f.). *Frequently Viewed or Downloaded: Top 100*. Consultado el 24 de septiembre de 2026, desde <https://www.gutenberg.org/browse/scores/top.html>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10.^a ed.). Wiley.
- Stix, G. (1991). Encoding the “Neatness” of Ones and Zeroes. *Scientific American*, 265(3), 54-58. <https://doi.org/10.1038/scientificamerican0991-54>
- The Linux man-pages Project. (2026, 8 de febrero). *pipe(2)* — *Linux manual page*. Consultado el 24 de septiembre de 2026, desde <https://man7.org/linux/man-pages/man2/pipe.2.html>

- The Linux man-pages Project. (s.f.-a). *fork(2)* — *Linux manual page*. Consultado el 24 de septiembre de 2026, desde <https://man7.org/linux/man-pages/man2/fork.2.html>
- The Linux man-pages Project. (s.f.-b). *open_memstream(3)* — *Linux manual page*. Consultado el 24 de septiembre de 2026, desde https://man7.org/linux/man-pages/man3/open_memstream.3.html
- The Linux man-pages Project. (s.f.-c). *pthread(7)* — *Linux manual page*. Consultado el 24 de septiembre de 2026, desde <https://man7.org/linux/man-pages/man7/pthreads.7.html>
- The Linux man-pages Project. (s.f.-d). *wait(2)* — *Linux manual page*. Consultado el 24 de septiembre de 2026, desde <https://man7.org/linux/man-pages/man2/wait.2.html>